CONVERSATIONAL ANALYTICS DASHBOARD

Technical Project Report

An AI Text-to-SQL Analytics System

React • Flask • MySQL • Ollama (local LLM)

Submitted by

HARSHIT VERMA

Source: github.com/Harshit262442/conversational-analytics

2026

CONTENTS

(In Word: click below, then References → Update Table to fill page numbers.)

CONTENTS 2

1. PROJECT OVERVIEW 4

   1.1 What it is 4

   1.2 Problem solved 4

   1.3 Capabilities at a glance 4

2. SYSTEM ARCHITECTURE 4

   2.1 Three-tier design 4

   2.2 Why this separation matters 5

3. TECHNOLOGY STACK 5

   3.1 Components and rationale 5

   3.2 Languages 5

4. DATABASE DESIGN 5

   4.1 Overview 5

   4.2 Tables by domain 5

   4.3 Notable design points 5

   4.4 How the data is created 6

5. BACKEND (FLASK) 6

   5.1 Responsibilities 6

   5.2 API endpoints 6

   5.3 The text-to-SQL pipeline (core of the project) 6

   5.4 Safety guard 7

   5.5 Automatic chart detection 7

6. THE AI / CHATBOT LAYER (OLLAMA) 7

   6.1 How the model is hosted 7

   6.2 Performance on a CPU 7

   6.3 Single call point 7

   6.4 Prompt engineering 7

   6.5 Insights and follow-ups flow 7

7. FRONTEND (REACT) 8

   7.1 Structure 8

   7.2 Main components 8

   7.3 State and API communication 8

   7.4 Visual design 8

8. KEY FEATURES 8

9. RUNNING THE SYSTEM 9
9.1 Local components 9
9.2 Start sequence 9
9.3 Version control 9
10. SECURITY AND PRIVACY 9
11. END-TO-END WALKTHROUGH 9
12. CHALLENGES AND SOLUTIONS 10
12.1 Misleading generic errors 10
12.2 Keeping the model accurate 10
12.3 Local inference speed 10
12.4 Keeping sample data current 10
APPENDIX: FULL DATABASE SCHEMA 11
Sales 11
Human Resources 11
Inventory 11
Purchasing 11
Manufacturing 12
System 12

1. PROJECT OVERVIEW

1.1 What it is

The Conversational Analytics Dashboard is a full-stack web application that lets a non-technical user query a company database by typing a question in plain English. A locally hosted large language model (LLM) converts the question into a SQL query, the backend safely executes it, and the result is rendered automatically as a chart, table, or single metric. The application spans five business domains — Sales, Human Resources, Inventory, Purchasing, and Manufacturing.

1.2 Problem solved

In most organisations, reading data from a database requires SQL knowledge, so business users depend on analysts for every report — slow and not scalable. This project removes that barrier: anyone can ask a question in their own words and get an instant, visualised answer.

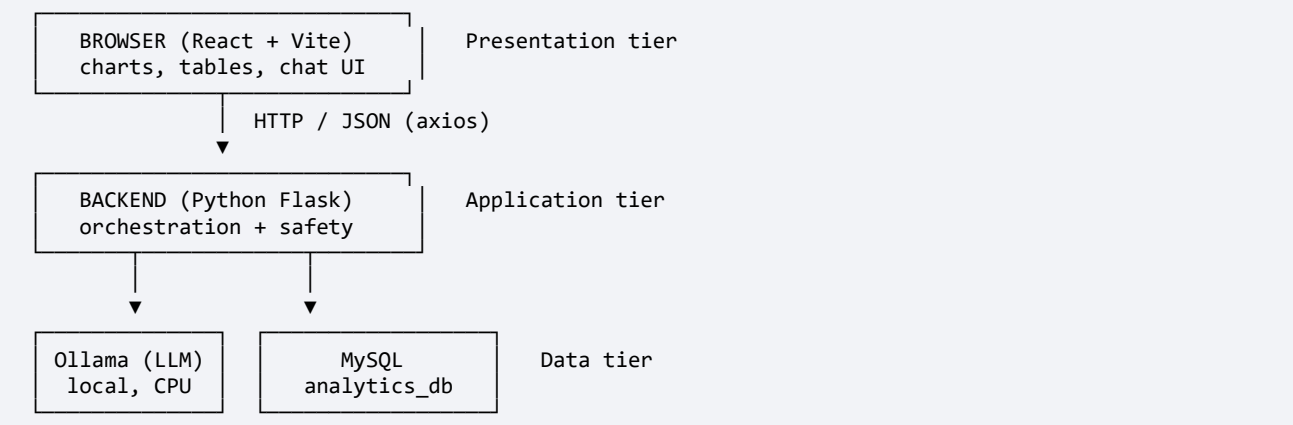
1.3 Capabilities at a glance

- Natural-language to SQL conversion using a local LLM, with the full schema supplied as context.
- Runs entirely on the machine via Ollama — offline, free, with no API keys, no usage limits, and no data leaving the computer.
- A safety layer allowing only read-only SELECT queries — never INSERT, UPDATE, DELETE or DROP.
- A self-healing retry: a failed query is sent back to the model for automatic correction.
- Automatic chart-type selection based on the shape of the result.
- AI-generated insights and follow-up question suggestions after every result.
- Welcome KPI dashboard, schema explorer, command palette, CSV/PNG export, and an audit log.

2. SYSTEM ARCHITECTURE

2.1 Three-tier design

The system follows a classic three-tier client–server architecture. The browser is the presentation tier, the Flask server is the application tier, and MySQL is the data tier. The AI model runs locally through Ollama and is called only by the backend.



The browser never talks to the LLM or the database directly. Every request flows through the Flask backend — the single place where input is validated, the AI is called, SQL is checked for safety, and results are shaped. This keeps security in one trusted location.

2.2 Why this separation matters

- Security: the SELECT-only guard lives server-side, so a malicious browser request cannot bypass it.
- Swappability: the LLM model, database engine, and chart library are each isolated and replaceable.
- Privacy: because the model runs locally, the question and the data never leave the machine.

3. TECHNOLOGY STACK

3.1 Components and rationale

Layer	Technology	Why chosen
Frontend	React 18 + Vite	Component model; Vite gives fast hot-reload and small builds
Charts	Recharts	Declarative, React-native charting
HTTP client	axios	Interceptors for centralised token + error handling
PNG export	html2canvas	Captures the rendered chart as an image
Backend	Python Flask 3	Lightweight, ideal for a small focused API
CORS	flask-cors	Controls which origins may call the API
LLM	Ollama + qwen2.5-coder	Local, free, offline; no API keys or quotas
Database	MySQL 8	Standard SQL, reviewable in MySQL Workbench
Hosting	Local machine	Runs fully on the user's computer

3.2 Languages

Python powers the backend for its first-class support of both web APIs (Flask) and AI integration. JavaScript with React powers the frontend. The two communicate over HTTP using JSON.

4. DATABASE DESIGN

4.1 Overview

The database is named analytics_db and contains 22 tables: 20 business tables across five domains, plus two system tables. It runs as MySQL on the local machine and is inspectable in MySQL Workbench. All time-based data is seeded relative to the current date (CURDATE() − INTERVAL n DAY) so that "this week" and "today" queries always return rows. The full column-level schema is in the Appendix.

4.2 Tables by domain

Domain	Tables
Sales	customers, sales_orders, sales_order_items, invoices
Human Resources	employees, attendance, leaves, payroll
Inventory	products, warehouses, inventory, stock_movements
Purchasing	purchase_orders, purchase_order_items, goods_receipts
Manufacturing	units_produced, defect_logs, machine_status, shift_records, suppliers
System	users (logins, SHA-256 hashes), query_log (audit trail)

4.3 Notable design points

- Relationships use foreign keys (e.g. sales_order_items.sales_order_id → sales_orders.id) so the LLM can JOIN correctly.
- A product's stock is split across warehouses; reorder checks SUM the quantity per product before comparing to reorder_level.
- query_log records the question, generated SQL, row count, chart type, user, and timestamp for every request.

4.4 How the data is created

Three scripts build and populate the database: seed.sql (manufacturing + system tables), seed_extended.sql (HR, Inventory, Sales, Purchase), and seed_users.py (SHA-256 password hashes for the sample logins). Re-running them refreshes all dates relative to the day they are run.

5. BACKEND (FLASK)

5.1 Responsibilities

The backend (a single app.py) is the brain of the system. It authenticates users, builds the AI prompt, calls the LLM, cleans and validates the returned SQL, executes it, detects the chart type, logs the query, and returns structured JSON. It also serves AI insights, follow-up suggestions, and dashboard KPIs.

5.2 API endpoints

Method + Path	Purpose
POST /api/login	Verify credentials, issue a session token
POST /api/logout	Invalidate the session token
POST /api/query	Convert question → SQL → execute → return result
GET /api/history	Last 20 questions from the audit log
POST /api/feedback	Mark a query as incorrect
GET /api/export/csv	Download a past result as CSV
GET /api/dashboard	Four KPI tiles for the welcome screen
POST /api/insights	AI observations about a result
POST /api/followups	AI-suggested next questions
GET /api/health	Liveness + active LLM model

5.3 The text-to-SQL pipeline (core of the project)

When a question arrives at POST /api/query, the backend runs these steps:

1. Build the system prompt — inject today's date and the full schema, plus strict output rules.
2. Call the local LLM with the system prompt and the user's question.
3. Clean the response — strip markdown, comments and prose; keep only the first SQL statement.
4. Validate safety — must start with SELECT; must not contain INSERT/UPDATE/DELETE/DROP/etc.
5. Execute against the database and capture columns + rows.
6. Self-healing retry — on failure, send the SQL and error back to the LLM for one corrected attempt.
7. Detect chart type from the result shape (metric / line / bar / table).
8. Log the question, SQL, row count, and chart type to query_log.
9. Return JSON: { sql, columns, rows, chart_type }.

5.4 Safety guard

Two regular expressions enforce read-only access independently of the model. Prompt-injection attempts such as "delete all customers" are rejected before reaching the database.

```
SELECT_ONLY = ^\s*select\b
FORBIDDEN   = \b(insert|update|delete|drop|alter|
               truncate|create|grant|replace)\b
is_safe = SELECT_ONLY.match(sql) and not FORBIDDEN.search(sql)
```

5.5 Automatic chart detection

Result shape	Chart chosen
1 row, 1 column	metric (big animated number)
First column is a date	line chart (time series)
2 columns, second is numeric	bar chart (category vs value)
Anything else	table (sortable rows)

6. THE AI / CHATBOT LAYER (OLLAMA)

6.1 How the model is hosted

The AI runs through Ollama, a local runtime that hosts open-source models on the user's machine and exposes an HTTP API at localhost:11434. The backend posts the prompt to /api/generate and reads back the model's text. This means the entire AI capability is free, offline, has no usage quotas, and keeps all data on the machine. The model used is qwen2.5-coder, a code-specialised model well suited to writing SQL.

6.2 Performance on a CPU

Because the test laptop has integrated graphics rather than a dedicated GPU, the model runs on the CPU. Two settings keep it responsive: keep_alive holds the model in RAM so it is not reloaded on every request (this alone cut a roughly 25-second cold call down to about 4 seconds), and num_predict caps the output length so the model stops as soon as the SQL is complete. A smaller 3-billion-parameter variant can be selected to roughly halve response time further.

6.3 Single call point

All LLM calls pass through one function, call_llm(), which sends the request to Ollama and applies retry-with-backoff on transient errors. Centralising the call in one place keeps the rest of the backend independent of how the model is hosted, and makes the model easy to change by editing a single environment setting.

6.4 Prompt engineering

Three prompts drive the AI features, each with its own persona:

- SQL prompt — a senior MySQL analyst that returns only a SELECT; includes the full schema and shaping rules (e.g. a date first for time series).
- Insights prompt — a business analyst that writes one or two plain-English observations, never SQL.
- Follow-ups prompt — suggests three short next questions for guided exploration.

Local models benefit from simple, direct prompts; the prompts were tuned so the model produces clean output, and server-side parsers strip any stray code or preamble before the text reaches the user.

6.5 Insights and follow-ups flow

A single question actually triggers three AI calls. The first writes the SQL and produces the chart and table, which are shown immediately. Two further calls — one for the insight and one for the follow-up questions — run afterwards in the background and stream in a moment later, so the user is never blocked waiting for them. Both are best-effort: if the model returns nothing useful, the parser discards it and the section simply does not render. Clicking a follow-up chip re-runs the whole pipeline with that question, turning one answer into a guided exploration.

7. FRONTEND (REACT)

7.1 Structure

The frontend is a single-page React application built with Vite. The top-level App component owns shared state (user session, history, current result, loading flags) and delegates rendering to focused child components, each receiving data via props and emitting actions via callbacks.

7.2 Main components

Component	Role
Login	Authentication form
HistoryPanel	Collapsible sidebar of past questions
ChatInput	Question entry + command-palette trigger
ChartRenderer	Selects and renders metric / line / bar
DataTable	Raw result rows
InsightsCard	AI-generated observations
FollowUps	Clickable next-question chips
WelcomeDashboard	KPI tiles on first load
SchemaExplorer	"What can I ask?" guide for users
CommandPalette	Ctrl+K fuzzy launcher
DataNetwork	Animated particle-network background

7.3 State and API communication

State is managed with React hooks (useState, useEffect, useRef) — no external state library is needed at this scale. All HTTP calls go through one axios module (api/client.js); the Vite dev server proxies the /api path to the local Flask server. A response interceptor catches expired sessions and signs the user out. After a result loads, the insights and follow-up requests are fired in parallel so they do not block each other.

7.4 Visual design

A restrained dark "glass-morphism" theme with subtle gradient accents, an animated data-network canvas as the signature background, and gentle transitions. The styling deliberately avoids over-animation so the product reads as professional software.

8. KEY FEATURES

Feature	Description
Natural-language query	Ask in English; SQL is executed and charted automatically

Feature	Description
Fully local AI	Runs offline through Ollama — no API keys, quotas, or data leaving the machine
Auto visualization	Metric / line / bar / table chosen from the data shape
AI insights	One or two plain-English observations per result
Follow-up suggestions	Three clickable next questions for guided exploration
Welcome dashboard	Revenue, low-stock, pending POs, units-today KPI tiles
Schema explorer	Plain-language guide to what can be asked
Command palette	Ctrl+K fuzzy search over history and suggestions
History + feedback	Audit log of every query; thumbs-down to flag wrong answers
Export	Download any result as CSV or the chart as PNG
Safe + self-healing	SELECT-only guard plus automatic SQL error correction

9. RUNNING THE SYSTEM

9.1 Local components

The application runs entirely on the local machine. Three things run together:

- MySQL — holds the analytics_db database.
- Ollama — hosts the language model and serves it at localhost:11434.
- The app servers — Flask (backend, port 5000) and the Vite dev server (frontend, port 5173).

9.2 Start sequence

The backend is started first (it connects to MySQL and to Ollama), then the frontend; the user opens the browser at the local address and signs in. Because everything is local, the application works with no internet connection.

9.3 Version control

The full source is kept in a Git repository on GitHub, so the project can be cloned, reviewed, and re-run on any machine that has MySQL and Ollama installed.

10. SECURITY AND PRIVACY

Risk	Mitigation
AI generates destructive SQL	Regex blocks anything that is not a SELECT
Direct unsafe API calls	Same guard runs server-side, not in the browser
SQL injection via the question	User text is sent to the LLM, never concatenated into SQL
Password exposure	Stored as SHA-256 hashes, never plaintext
Data leaving the machine	The model is local — questions and data never go to any cloud service
Unrestricted cross-origin	CORS limited to the known frontend origin

Known limitations and next steps: the login is currently a UI gate (a production build would enforce JWT sessions and use bcrypt instead of SHA-256). This is documented as a planned improvement.

11. END-TO-END WALKTHROUGH

One sentence that captures the whole system: the user types an English question in the React frontend; it is sent to the Flask backend, which forwards it with the database schema to the local Ollama model; the model returns SQL; the backend validates that it is a safe SELECT, runs it on the MySQL database, auto-selects a chart, logs it, and returns the rows; the frontend renders the chart, table, an AI insight, and three follow-up suggestions.

The chart and table appear first, from the SQL-writing call. Two further model calls then produce the insight and the follow-up questions in the background, so one query becomes a small guided analysis without making the user wait. If the SQL ever fails, the backend asks the model to fix it once before showing any error, which is what makes the system feel reliable.

12. CHALLENGES AND SOLUTIONS

12.1 Misleading generic errors

A generic "Request failed" message once hid the real cause — a missing database column returning an HTML error page. Lesson applied: surface JSON-shaped errors and check backend logs first.

12.2 Keeping the model accurate

The local model occasionally wrote SQL when asked for prose, or compared per-warehouse stock instead of the total. Fixes: simpler, more directive prompts; explicit SUM-across-warehouses guidance in the schema; and parsers that strip any stray code from the insight text.

12.3 Local inference speed

On a CPU-only laptop a 7-billion-parameter model is heavy. Keeping the model warm in RAM (keep_alive) cut a roughly 25-second cold call to about 4 seconds, and capping the output length (num_predict) avoids the model rambling. A smaller 3b model can halve the time again.

12.4 Keeping sample data current

Time-based questions need recent data. All dated rows are seeded relative to the current date, so re-running the seed scripts refreshes the data and "this week" / "today" queries always return results.

APPENDIX: FULL DATABASE SCHEMA

Every table with its columns. Notation: PK = primary key; → marks a foreign key; [..] lists allowed values.

Sales

customers

```
id PK, customer_name, email, phone, region [North/South/East/West], segment [enterprise/mid_market/smb], created_at
```

sales_orders

```
id PK, order_number, customer_id → customers.id, order_date, status [pending/shipped/delivered/cancelled], total_amount, salesperson
```

sales_order_items

```
id PK, sales_order_id → sales_orders.id, product_id → products.id, quantity, unit_price, line_total
```

invoices

```
id PK, invoice_number, sales_order_id → sales_orders.id, invoice_date, amount, status [paid/pending/overdue], paid_on
```

Human Resources

employees

```
id PK, employee_code, full_name, email, department, role, hire_date, salary, manager_id → employees.id, status [active/on_leave/resigned]
```

attendance

```
id PK, employee_id → employees.id, work_date, check_in TIME, check_out TIME, hours_worked, status [present/absent/half_day/wfh]
```

leaves

```
id PK, employee_id → employees.id, leave_type [sick/casual/earned/unpaid], start_date, end_date, status [approved/pending/rejected], reason
```

payroll

```
id PK, employee_id → employees.id, pay_month (first of month), basic, allowances, deductions, net_pay, paid_on
```

Inventory

products

```
id PK, sku, product_name, category [Raw Material/Tooling/Component/Consumable/Finished Goods/Spare Part/PPE], unit_price, reorder_level
```

warehouses

```
id PK, warehouse_name, location, capacity
```

inventory

```
id PK, product_id → products.id, warehouse_id → warehouses.id, quantity, last_updated DATETIME
```

stock_movements

```
id PK, product_id → products.id, warehouse_id → warehouses.id, movement_type [in/out/transfer/adjust], quantity, reference, movement_date DATETIME
```

Purchasing

purchase_orders

```
id PK, po_number, supplier_id → suppliers.id, order_date, status [pending/received/cancelled], total_amount, buyer
```

purchase_order_items

```
id PK, po_id → purchase_orders.id, product_id → products.id, quantity, unit_cost, line_total
```

goods_receipts

```
id PK, po_id → purchase_orders.id, received_date, received_by, status [complete/partial/rejected]
```

Manufacturing

units_produced

```
id PK, machine_id, shift_date, shift_type [morning/evening/night], units_count, operator_name
```

defect_logs

```
id PK, machine_id, defect_type, severity [low/medium/high/critical], detected_at DATETIME, supplier_id → suppliers.id, units_affected
```

machine_status

```
id PK, machine_id, machine_name, status [running/maintenance/down], last_maintenance, department
```

shift_records

```
id PK, shift_date, shift_type, operator_name, machine_id, hours_worked
```

suppliers

```
id PK, supplier_name, region, contact_email, rating
```

System

users

```
id PK, username, password_hash (SHA-256), department
```

query_log

```
id PK, question, generated_sql, row_count, was_correct, username, department, chart_type, created_at
```